Osnove razvoja web i mobilnih aplikacija · Laboratorijske vježbe

LV3
Uvod u JavaScript

Varijable, funkcije, DOM i događaji

Fakultet elektrotehnike, računarstva i informacijskih tehnologija Osijek
Kneza Trpimira 2b · www.ferit.unios.hr

Sadržaj

1. Uvod
2. Prvi primjer
3. JavaScript u HTML dokumentu
4. Konzola u pregledniku
5. Varijable i tipovi podataka
6. Grananje
7. Funkcije
8. Nizovi
9. Objekti
10. Petlje
11. Manipulacija DOM-om
12. Događaji i classList
13. Primjeri
14. Frontend i backend
15. Zadaci

Prilog: početni predlošci

1. Uvod

JavaScript je programski jezik weba. Izvršava se u svakom web pregledniku i stranici daje **ponašanje**: reagira na korisnika i mijenja sadržaj bez ponovnog učitavanja stranice. HTML opisuje strukturu stranice, CSS njezin izgled, a JavaScript interakciju.

- **Dinamički tipovi:** tip se ne piše uz varijablu, nego ovisi o vrijednosti koju ona trenutno sadrži.
- **Izvršavanje:** kod se ne prevodi unaprijed u izvršnu datoteku. Preglednik kod prevodi u strojni kod tijekom izvođenja (JIT, *just in time*).
- **ECMAScript:** standardizirana inačica jezika (Ecma International). Verzija ES6 iz 2015. uvela je `let`, `const` i arrow funkcije, koje koristimo u cijeloj vježbi.
- Osim u pregledniku, JavaScript se koristi i na serveru (Node.js) te za desktop aplikacije (Electron).

2. Prvi primjer

Cijela vježba temelji se na tri koraka: **pronađi element**, **čekaj događaj**, **promijeni stranicu**. Sljedeći primjer mijenja tekst naslova kad korisnik klikne gumb.

index.html

```
<h1 id="pozdrav">Pozdrav!</h1>
<button id="gumb">Klikni me</button>
```

script.js

```
const naslov =
document.querySelector("#pozdrav");
const gumb =
document.querySelector("#gumb");

gumb.addEventListener("click", () => {
  naslov.textContent = "Kliknuo si gumb!";
});
```

Isto bi se moglo napisati atributom `onclick="…"` u HTML-u, ali u ovoj vježbi događaje dodajemo metodom `addEventListener`, kako bi HTML i JavaScript ostali odvojeni.

3. JavaScript u HTML dokumentu

Način	Kod	Napomena
U head s defer (preporučeno)	`<script src="script.js" defer></script>`	Skripta se izvrši tek kad je cijela stranica učitana.
Na kraju body	`<script src="script.js"></script>` prije `</body>`	Također radi: skripta vidi sve elemente iznad sebe.
Unutar dokumenta	`<script> … </script>`	Samo za kratke primjere.

> Bez atributa `defer` skripta u `<head>` izvrši se prije nego što elementi stranice postoje, pa `querySelector` vrati `null`. Datoteka `.js` sadrži samo JavaScript, **bez** oznaka `<script>`. Vrijednosti atributa u HTML-u pišu se običnim navodnicima `"`, ne „pametnim” navodnicima iz uređivača teksta.

Preporučena struktura projekta, kao u prvoj vježbi:

```
lv3/
├── index.html
├── css/
│   └── style.css
└── js/
    └── script.js
```

4. Konzola u pregledniku

Konzola se otvara tipkom **F12** (ili **Ctrl + Shift + I**), na kartici **Console**. U njoj se vidi sve što ispiše `console.log` i svaka greška. Kad nešto ne radi, prvo pogledajte konzolu i ispišite vrijednost koju sumnjate: `console.log(a, typeof a)`. U konzolu se može i izravno upisati JavaScript.

Primjer poruke o grešci

```
Uncaught ReferenceError: rezultatt is not defined        script.js:7
```

Dio poruke	Značenje
`ReferenceError`	vrsta greške: korištena je varijabla koja ne postoji
`rezultatt is not defined`	o kojem je imenu riječ (ovdje tipfeler)
`script.js:7`	datoteka i redak; klik vodi na mjesto greške

Česte greške: `TypeError: Cannot read properties of null` (selektor nije pronašao element ili skripta nema `defer`), `SyntaxError` (nedostaje zagrada ili navodnik).

5. Varijable i tipovi podataka

5.1 Osnovni tipovi

Tip	Vrijednosti	Primjer
number	svi brojevi, cijeli i decimalni	`42`, `3.5`; `7 / 2` daje 3.5
string	tekst u navodnicima	`"Ana"`, `'Ana'`, `` `Ana` ``
boolean	istina ili laž	`true`, `false`
undefined	varijabla još nema vrijednost	`let x;`
null	namjerno prazna vrijednost	`let x = null;`

Tip se ne piše uz varijablu: određuje ga vrijednost koju varijabla trenutno sadrži. Složeni tipovi su **nizovi** i **objekti** (poglavlja 8 i 9). Operator `typeof` vraća ime tipa: `typeof "Ana"` daje `"string"`.

5.2 Deklaracija: const i let

let vrijedi samo unutar bloka { }

```
let a = 10;
function f() {
  if (true) {
    let b = 9;
    console.log(b);   // 9
  }
  console.log(b);     // ReferenceError
}
```

const se ne može ponovno pridružiti

```
const PI = 3.14;
PI = 3;   // TypeError: Assignment
          // to constant variable.

let ime = "Ana";      // string
const godine = 21;    // number
let student = true;   // boolean
```

	const	let	var
Opseg	blok { }	blok { }	funkcija
Nova vrijednost	ne	da	da
Kada koristiti	uvijek kad može	kad se vrijednost mijenja	samo u starom kodu

`var` se pojavljuje u starijim primjerima na internetu. Ponaša se drukčije (opseg je cijela funkcija), pa ga u novom kodu ne koristimo.

5.3 Usporedba: == i ===

```
console.log("5" == 5);    // true
console.log("5" === 5);   // false
console.log(0 == "");     // true
console.log(0 === "");    // false
```

Operator `==` prije usporedbe pretvara tipove i zato daje neočekivane rezultate. Operator `===` uspoređuje i vrijednost i tip, pa ga **koristimo uvijek**. Isto vrijedi za `!=` i `!==`.

6. Grananje

```
let bodovi = 85;

if (bodovi >= 90) {
  console.log("Izvrstan");
} else if (bodovi >= 80) {
  console.log("Vrlo dobar");   // ispiše se ovo
} else {
  console.log("Dobar");
}
```

Uvjeti se provjeravaju redom i izvrši se samo prvi blok čiji je uvjet istinit. Više uvjeta spaja se logičkim operatorima:

Operator	Značenje	Primjer
`&&`	i: oba uvjeta istinita	`bodovi >= 80 && bodovi < 90`
`||`	ili: barem jedan istinit	`ime === "" || email === ""`
`!`	ne: obrće uvjet	`!prijavljen`

7. Funkcije

Funkcija s ključnom riječi function

```
function pozdravi(ime) {
  console.log('Pozdrav, ' + ime + '!');
}
pozdravi('Ana');   // Pozdrav, Ana!
```

Arrow funkcija

```
const pozdravi = (ime) => {
  console.log(`Pozdrav, ${ime}!`);
};
const kvadrat = (x) => x * x;
kvadrat(4);        // 16
```

Arrow funkcija je **kraći zapis** funkcije. Najčešće je pišemo kad funkciju predajemo drugoj funkciji, npr. u `addEventListener` ili `forEach`. Ako tijelo ima samo jedan izraz, vitičaste zagrade i `return` mogu se izostaviti. Znakovi `` ` `` (backtick) omogućuju umetanje vrijednosti u tekst s `${…}`.

8. Nizovi

Niz čuva više vrijednosti pod jednim imenom. Piše se u uglatim zagradama, a elementi se odvajaju zarezom. Svaki element ima **indeks**, redni broj koji počinje od **0**.

8.1 Pristup i promjena elemenata

```
const voce = ["jabuka", "banana", "kruška"];

console.log(voce[0]);       // jabuka
console.log(voce.length);   // 3
console.log(voce[voce.length - 1]);  // kruška

voce[1] = "šljiva";
console.log(voce);
// ["jabuka", "šljiva", "kruška"]
console.log(voce[5]);       // undefined
```

- `voce[0]` je prvi element, a `voce[voce.length - 1]` zadnji.
- `length` je broj elemenata.
- Element se mijenja pridruživanjem: `voce[1] = "šljiva"`.
- Nepostojeći indeks daje `undefined`, a ne grešku.
- Niz deklariran s `const` i dalje se može mijenjati iznutra; `const` samo sprječava da varijabli pridružimo drugi niz.
- Niz može sadržavati vrijednosti različitih tipova.

8.2 Metode nizova

```
const brojevi = [3, 7];
brojevi.push(10);          // [3, 7, 10]
brojevi.unshift(1);        // [1, 3, 7, 10]
brojevi.pop();             // [1, 3, 7]
brojevi.shift();           // [3, 7]

brojevi.includes(7);       // true
brojevi.indexOf(7);        // 1
brojevi.indexOf(99);       // -1
brojevi.join(" - ");       // "3 - 7"
```

Metoda	Djelovanje
`push(x)`	dodaje x na kraj
`pop()`	uklanja i vraća zadnji element
`unshift(x)`	dodaje x na početak
`shift()`	uklanja i vraća prvi element
`includes(x)`	true ako niz sadrži x
`indexOf(x)`	indeks od x, ili -1 ako ga nema
`join(s)`	spaja elemente u tekst, odvojene sa s

9. Objekti

Objekt opisuje **jednu stvar** pomoću parova **ključ: vrijednost**. Piše se u vitičastim zagradama. Vrijednost može biti bilo kojeg tipa, pa i niz, drugi objekt ili funkcija.

9.1 Rad sa svojstvima

```
const student = {
  ime: "Ana",
  godina: 2,
  smjer: "Računarstvo"
};

console.log(student.ime);         // Ana
console.log(student["smjer"]);    // Računarstvo
student.godina = 3;               // promjena
student.email = "ana@primjer.hr"; // novo svojstvo
delete student.smjer;             // brisanje
```

- Svojstvu se pristupa **točkom**: `student.ime`.
- **Uglate zagrade** koristimo kad je ime svojstva u varijabli ili sadrži razmak: `student[kljuc]`.
- Pridruživanje mijenja postojeće svojstvo ili dodaje novo; `delete` ga briše.
- Svojstvo čija je vrijednost funkcija zove se **metoda**: `auto.pokreni()`.

Objekt s metodom

```
let auto = {
  marka: "Toyota",
  pokreni: function () {
    console.log("Auto je upaljen");
  }
};
auto.pokreni();   // Auto je upaljen
```

9.2 Niz objekata

```
const zadaci = [
  { naziv: "Kupiti kruh", gotovo: false },
  { naziv: "Riješiti LV3", gotovo: true }
];

console.log(zadaci[1].naziv);   // Riješiti LV3
zadaci.push({
  naziv: "Nazvati baku",
  gotovo: false
});
console.log(zadaci.length);     // 3
```

Podaci aplikacije najčešće su **niz objekata**: svaki element niza opisuje jednu stavku, a njegova svojstva dijelove te stavke. Tako se može spremiti, primjerice, lista zadataka.

`zadaci[1].naziv` čita se s lijeva na desno: najprije element niza s indeksom 1, zatim njegovo svojstvo `naziv`.

10. Petlje

Petlja ponavlja blok koda. Svaka petlja treba uvjet koji jednom postane lažan; inače se nikad ne zaustavi i stranica se zamrzne.

Petlja	Prolazi kroz...
`for`	blok koda zadani broj puta, uz brojač
`while`	blok koda dok je uvjet istinit
`do...while`	isto, ali se blok izvrši barem jednom
`for...of`	vrijednosti niza ili znakove stringa
`for...in`	ključeve (svojstva) objekta
`forEach()`	metoda niza: poziva funkciju za svaki element

10.1 Petlja for

```
const ocjene = [5, 3, 4];
let zbroj = 0;

for (let i = 0; i < ocjene.length; i++) {
  zbroj += ocjene[i];
}
console.log(zbroj / ocjene.length);  // prosjek: 4
```

Dio	Uloga
`let i = 0`	početak: izvrši se jednom
`i < ocjene.length`	uvjet: provjerava se prije svakog kruga
`i++`	korak: izvrši se nakon svakog kruga

Krug	i	ocjene[i]	zbroj nakon kruga
1.	0	5	5
2.	1	3	8
3.	2	4	12
kraj	3	uvjet 3 < 3 je lažan	12

10.2 Petlje while i do...while

```
let n = 1;
while (n < 100) {
  n = n * 2;
}
console.log(n);   // 128
```

```
let i = 10;
do {
  console.log("izvrši se barem jednom");
  i++;
} while (i < 5);
```

`while` koristimo kad broj ponavljanja nije unaprijed poznat. Kod `do...while` uvjet se provjerava na kraju, pa se tijelo izvrši barem jednom, iako je ovdje uvjet od početka lažan.

10.3 for...of i for...in

for...of: vrijednosti niza

```
const boje = ["crvena", "zelena", "plava"];
for (const boja of boje) {
  console.log(boja);
}
// crvena, zelena, plava
```

for...in: ključevi objekta

```
const auto = { marka: "Toyota", godina: 2020 };
for (const kljuc in auto) {
  console.log(kljuc, auto[kljuc]);
}
// marka Toyota, godina 2020
```

Za nizove koristimo `for...of`. `for...in` na nizu daje indekse kao tekst (`"0"`, `"1"`...), a ne vrijednosti.

10.4 Metoda forEach()

```
const brojevi = [45, 4, 9];

brojevi.forEach((broj, indeks) => {
  console.log(indeks, broj);
});

let zbroj = 0;
brojevi.forEach((broj) => {
  zbroj += broj;
});
console.log("zbroj:", zbroj);   // 58
```

`forEach` poziva zadanu funkciju jednom za svaki element niza i predaje joj vrijednost, indeks i cijeli niz (potrebne parametre navodimo redom). Za razliku od `for` i `for...of`, ne može se prekinuti naredbom `break`.

10.5 Koju petlju odabrati

Isti zadatak na tri načina

```
for (let i = 0; i < boje.length; i++) console.log(boje[i]);
for (const boja of boje) console.log(boja);
boje.forEach((boja) => console.log(boja));
```

Potrebno	Petlja
samo vrijednosti niza	`for...of` ili `forEach`
indeks, drugačiji korak ili prekid s break	`for`
ključevi i vrijednosti objekta	`for...in`
broj ponavljanja nije unaprijed poznat	`while`

11. Manipulacija DOM-om

Kad se stranica učita, preglednik napravi **DOM** (Document Object Model), stablo objekata: `document` → `html` → `head` i `body` → elementi → tekst. Preko DOM-a JavaScript može mijenjati, dodavati i brisati elemente, atribute i stilove te reagirati na događaje. **Metode** su akcije nad elementima, a **svojstva** vrijednosti koje čitamo ili mijenjamo.

11.1 Dohvat elemenata

```
document.getElementById("demo")        // element s id="demo"
document.querySelector("#demo")        // prvi element koji odgovara CSS selektoru
document.querySelector(".stavka")
document.querySelectorAll("li")       // svi elementi koji odgovaraju

const p = document.querySelector("#demo");
p.textContent = "Hello World!";        // promjena sadržaja
slika.src = "landscape.jpg";           // promjena atributa
poruka.style.display = "none";         // promjena stila
```

`querySelector` prima isti selektor kao CSS. Ako ništa ne pronađe, vraća `null`.

11.2 Čitanje unosa

```
const a = document.querySelector("#a").value;  // "5"
const b = document.querySelector("#b").value;  // "3"

console.log(a + b);                  // "53": spajanje teksta!
console.log(Number(a) + Number(b));  // 8
console.log(isNaN(Number("abc")));   // true
```

Izraz	Značenje
`input.value`	uvijek string, i za `type="number"`
`Number(x)`	pretvara u broj; neispravan unos daje `NaN`
`isNaN(x)`	provjerava je li vrijednost NaN
`value.trim()`	uklanja razmake na početku i kraju; prazan unos postaje `""`

Operator `+` sa stringovima spaja tekst, zato "5" i "3" daju "53". Prije računanja vrijednosti iz polja uvijek pretvorite s `Number()`.

11.3 textContent ili innerHTML

```
const unos = "<b>Ana</b>";

prvi.textContent = unos;   // prikaže: <b>Ana</b>
drugi.innerHTML = unos;    // prikaže: Ana (podebljano)
```

`textContent` upisuje točno zadani tekst; oznake se prikažu kao znakovi. `innerHTML` tekst čita kao HTML. Za tekst koji upisuje korisnik koristite **textContent**: s `innerHTML` netko može na stranicu ubaciti vlastiti HTML ili skriptu.

11.4 Dodavanje i uklanjanje elemenata

```
const p = document.createElement("p");   // stvori element
p.textContent = "Ovo je novi paragraf";
document.body.appendChild(p);           // dodaj ga kao zadnje dijete

p.remove();                             // ukloni element
roditelj.removeChild(dijete);           // ili preko roditelja
```

12. Događaji i classList

```
const lista =
document.querySelector("ul");

lista.addEventListener("click", (e) => {
  console.log(e.target);
});
```

`addEventListener` povezuje događaj s funkcijom, a `removeEventListener` ga uklanja. Funkcija prima **objekt događaja** `e`: `e.target` je element na koji je kliknuto, pa jedan slušatelj na listi pokriva sve stavke. `e.preventDefault()` sprječava zadano ponašanje (npr. slanje forme), a `e.stopPropagation()` sprječava da klik stigne do roditelja.

Događaj	Kada se dogodi
`click`	korisnik klikne element
`input`	promijeni se sadržaj polja
`change`	polje je promijenjeno i napušteno
`keydown`	pritisnuta je tipka
`mouseover`	pokazivač dođe na element
`submit`	forma se šalje
`DOMContentLoaded`	HTML stranice je učitan

Kao HTML atribut događaj ima prefiks *on* (`onclick`), a u `addEventListener` piše se bez njega (`"click"`).

12.1 Mijenjanje izgleda pomoću klasa

style.css

```
.gotovo {
  text-decoration: line-through;
  color: gray;
}
```

script.js

```
lista.addEventListener("click", (e) => {
  e.target.classList.toggle("gotovo");
});
```

Metoda	Djelovanje
`classList.add("x")`	dodaje klasu
`classList.remove("x")`	uklanja klasu
`classList.toggle("x")`	dodaje ako je nema, uklanja ako postoji
`classList.contains("x")`	vraća true ili false

Izgled se piše u CSS-u kao klasa, a JavaScript samo dodaje ili uklanja klasu. Tako stil ostaje na jednom mjestu.

13. Primjeri

13.1 Brojač klikova

index.html

```
<p>Broj klikova: <span id="count">0</span>
</p>
<button id="plus">Dodaj</button>
```

style.css

```
button {
  padding: 10px 20px;
  background: #ff6600;
  color: white;
  border: none;
  border-radius: 5px;
}
```

script.js

```
let count = 0;
const prikaz =
  document.querySelector("#count");

document.querySelector("#plus")
  .addEventListener("click", () => {
    count++;
    prikaz.textContent = count;
  });
```

Varijabla `count` je izvan funkcije pa zadržava vrijednost između klikova.

13.2 Provjera forme

index.html

```
<form>
  <input type="text" id="ime" placeholder="Ime">
  <input type="email" id="email" placeholder="E-mail">
  <button type="submit">Pošalji</button>
  <p id="greska" class="greska">Ispunite sva polja.</p>
</form>
```

script.js

```
const forma = document.querySelector("form");
const greska = document.querySelector("#greska");

forma.addEventListener("submit", (e) => {
  const ime = document.querySelector("#ime").value.trim();
  const email = document.querySelector("#email").value.trim();
  if (ime === "" || email === "") {
    e.preventDefault();
    greska.style.display = "block";
  }
});
```

Poruka o grešci je u CSS-u skrivena (`.greska { display: none; color: red; }`). Kad podaci nisu ispravni, `e.preventDefault()` zaustavlja slanje forme i poruka se prikaže.

13.3 Svijetli i tamni način

style.css

```
.dark-mode {
  background-color: #333;
  color: white;
}
```

script.js

```
document.querySelector("#tema")
  .addEventListener("click", () => {
    document.body.classList.toggle("dark-
mode");
  });
```

14. Frontend i backend

	Frontend	Backend
Fokus	korisničko sučelje (UI) i iskustvo (UX)	serverska logika, baze podataka, prijava korisnika
Tehnologije	HTML, CSS, JavaScript; React, Vue, Angular	Node.js, Python, PHP, Java, C#; MySQL, PostgreSQL, MongoDB; Apache, Nginx
Izvršava se	u pregledniku korisnika	na serveru, nevidljivo korisniku
Primjeri	gumbi, slike, forme	pohrana podataka, poslovna logika, sigurnost

Frontend s backendom komunicira preko **API-ja**, najčešće REST ili GraphQL. U ovoj vježbi radimo samo frontend.

15. Zadaci

Za oba zadatka koristite početne predloške iz priloga. HTML, CSS i JavaScript držite u zasebnim datotekama, a projekt organizirajte kao u prvoj vježbi.

Zadatak 1: kalkulator

1. Prema predlošku napravite formu: dva polja `type="number"`, četiri gumba (+ − × ÷) i element za rezultat.
2. Dohvatite elemente s `querySelector` i na svaki gumb dodajte `addEventListener`.
3. Pročitajte vrijednosti, pretvorite ih s `Number()` i provjerite ih ispisom u konzolu.
4. Izračunajte i ispišite rezultat s `textContent` ispod kalkulatora.
5. Ako je polje prazno ili broj neispravan (`isNaN`) te kod dijeljenja s nulom, ispišite poruku umjesto rezultata.

Zadatak 2: lista zadataka (to-do lista)

1. Prema predlošku napravite polje za unos, gumb *Dodaj* i praznu listu `<ul>`. Na klik gumba pročitajte unos i ispišite ga u konzolu.
2. Stvorite novi `<li>` s upisanim tekstom (`createElement`, `textContent`, `appendChild`) i ispraznite polje.
3. Prazan unos (nakon `trim()`) ne smije dodati stavku.
4. Klik na zadatak označava ga obavljenim: `classList.toggle("gotovo")`. Klasa u CSS-u precrtava tekst ili mijenja boju.
5. Dodajte gumb *X* u svaku stavku koji briše zadatak (`remove()`). Klik na *X* ne smije označiti zadatak: koristite `e.stopPropagation()`.

Prilog: početni predlošci

Zadatak 1: index.html

```
<!DOCTYPE html>
<html lang="hr">
<head>
  <meta charset="utf-8">
  <title>Kalkulator</title>
  <link rel="stylesheet" href="css/style.css">
  <script src="js/script.js" defer></script>
</head>
<body>
  <main class="kalkulator">
    <h1>Kalkulator</h1>
    <input type="number" id="a" placeholder="Prvi broj">
    <input type="number" id="b" placeholder="Drugi broj">
    <div class="gumbi">
      <button id="zbroji">+</button>
      <button id="oduzmi">-</button>
      <button id="pomnozi">×</button>
      <button id="podijeli">÷</button>
    </div>
    <p id="rezultat"></p>
  </main>
</body>
</html>
```

Zadatak 2: index.html

```
<!DOCTYPE html>
<html lang="hr">
<head>
  <meta charset="utf-8">
  <title>Lista zadataka</title>
  <link rel="stylesheet" href="css/style.css">
  <script src="js/script.js" defer></script>
</head>
<body>
  <main class="todo">
    <h1>Lista zadataka</h1>
    <input type="text" id="unos" placeholder="Novi zadatak">
    <button id="dodaj">Dodaj</button>
    <ul id="lista"></ul>
  </main>
</body>
</html>
```